

```
# data sources:https://www.kaggle.com/datasets/adarshsng/lending-club-loan-data-csv
```

Double-click (or enter) to edit

```
from google.colab import files
files.upload()
```



选择文件

kaggle.json

- **kaggle.json**(application/json) – 74 bytes, last modified: 2025/6/20 – 100% done

Saving kaggle.json to kaggle.json

```
import os
os.makedirs("/root/.kaggle", exist_ok=True)
!mv kaggle.json /root/.kaggle/
!chmod 600 /root/.kaggle/kaggle.json
```

```
!kaggle datasets download -d adarshsng/lending-club-loan-data-csv
```

 Dataset URL: <https://www.kaggle.com/datasets/adarshsng/lending-club-loan-data-csv>

License(s): DbCL-1.0

lending-club-loan-data-csv.zip: Skipping, found more recently modified local copy (use --force to force download)

```
import zipfile
with zipfile.ZipFile("/content/lending-club-loan-data-csv.zip","r") as Lending_club:
    Lending_club.extractall("lending-club-loan-data-csv")
```

```
from google.colab import drive
drive.mount("/content/drive")
```

 Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_r

```
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from tabulate import tabulate
```

```
lending_club = pd.read_csv("/content/lending-club-loan-data-csv/loan.csv")
lending_club.head()
```

 /tmp/ipython-input-8-970121203.py:1: DtypeWarning: Columns (19,47,55,112,123,124,125,128,129,130,133,139,140,141) h

lending_club = pd.read_csv("/content/lending-club-loan-data-csv/loan.csv")

	id	member_id	loan_amnt	funded_amnt	funded_amnt_inv	term	int_rate	installment	grade	sub_grade	...	hard:
0	NaN	NaN	2500	2500	2500.0	36 months	13.56	84.92	C	C1	...	
1	NaN	NaN	30000	30000	30000.0	60 months	18.94	777.23	D	D2	...	
2	NaN	NaN	5000	5000	5000.0	36 months	17.97	180.69	D	D1	...	
3	NaN	NaN	4000	4000	4000.0	36 months	18.94	146.51	D	D2	...	
4	NaN	NaN	30000	30000	30000.0	60 months	16.14	731.78	C	C4	...	

5 rows x 145 columns

```
lending_club.isnull().sum()
```



	0
id	2260668
member_id	2260668
loan_amnt	0
funded_amnt	0
funded_amnt_inv	0
...	...
settlement_status	2227612
settlement_date	2227612
settlement_amount	2227612
settlement_percentage	2227612
settlement_term	2227612

145 rows × 1 columns

```
thresh = len(lending_club) * 0.7
lending_club = lending_club.dropna(thresh=thresh, axis=1)
lending_club = lending_club.dropna()
lending_club.head()
```



	loan_amnt	funded_amnt	funded_amnt_inv	term	int_rate	installment	grade	sub_grade	emp_title	emp_length
0	2500	2500	2500.0	36 months	13.56	84.92	C	C1	Chef	10+ years
1	30000	30000	30000.0	60 months	18.94	777.23	D	D2	Postmaster	10+ years
2	5000	5000	5000.0	36 months	17.97	180.69	D	D1	Administrative	6 years
3	4000	4000	4000.0	36 months	18.94	146.51	D	D2	IT Supervisor	10+ years
4	30000	30000	30000.0	60 months	16.14	731.78	C	C4	Mechanic	10+ years

5 rows × 87 columns

```
lending_club.nunique()
```



	0
loan_amnt	1561
funded_amnt	1561
funded_amnt_inv	1603
term	2
int_rate	356
...	...
total_bc_limit	9777
total_il_high_credit_limit	186578
hardship_flag	2
disbursement_method	2
debt_settlement_flag	2

87 rows × 1 columns

```
lending_club['loan_status'].value_counts()
```



loan_status	count
Fully Paid	764818
Current	671801
Charged Off	190439
Late (31-120 days)	15965
In Grace Period	6768
Late (16-30 days)	2635
Default	25

```
import xgboost as xgb
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
```

```
lending_club = lending_club.copy()
y=lending_club["loan_status"]
x= lending_club.drop(columns = ["loan_status"])
le = LabelEncoder()
```

```
for col in x.select_dtypes(include='object').columns:
    x[col] = LabelEncoder().fit_transform(x[col].astype(str))
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
x_train, x_val, y_train, y_val = train_test_split(x_train, y_train, test_size=0.25, random_state=42)
```

```
le = LabelEncoder()
y_train_encoded = le.fit_transform(y_train)
model = xgb.XGBClassifier(max_depth=6,n_estimators = 100,use_label_encoder=False,eval_metric = "logloss")
model.fit(x_train, y_train_encoded)
```

```

/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [22:43:15] WARNING: /workspace/src/learn
Parameters: { "use_label_encoder" } are not used.

```

```
warnings.warn(msg, UserWarning)
```

```

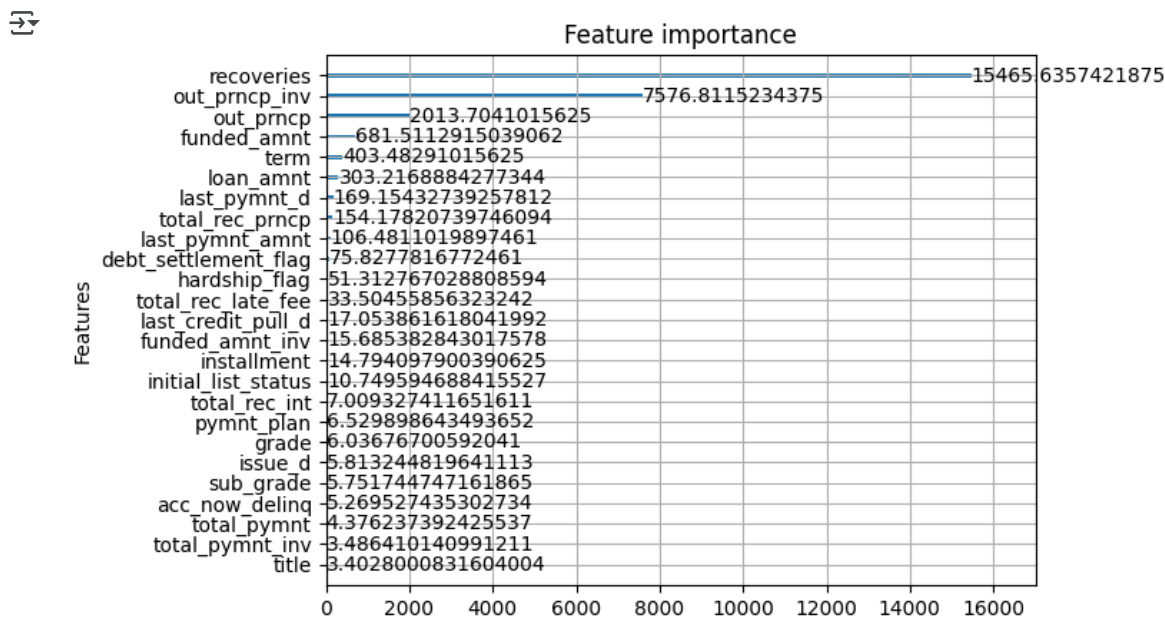
XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric='logloss',
               feature_types=None, gamma=None, grow_policy=None,
               importance_type=None, interaction_constraints=None,
               learning_rate=None, max_bin=None, max_cat_threshold=None,
               max_cat_to_onehot=None, max_delta_step=None, max_depth=6,
               max_leaves=None, min_child_weight=None, missing=nan,
               monotone_constraints=None, multi_strategy=None, n_estimators=100,
               n_jobs=None, num_parallel_tree=None, objective='multi:softprob', ...)

```

```

import matplotlib.pyplot as plt
xgb.plot_importance(model, max_num_features=25, importance_type='gain')
plt.show()

```



```

from sklearn.feature_selection import RFE
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
y_train_encoded = le.fit_transform(y_train)

gain_dict = model.get_booster().get_score(importance_type='gain')
sorted_features = sorted(gain_dict.items(), key=lambda x: x[1], reverse=True)
top_15 = [f[0] for f in sorted_features[:15]]
x_train_top15 = x_train[top_15]

rfe = RFE(estimator=model, n_features_to_select=10, step=1)
rfe.fit(x_train_top15, y_train_encoded)

selected_10 = x_train_top15.columns[rfe.support_]
print("The most important 10 features", selected_10.tolist())

```

```

/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [22:46:57] WARNING: /workspace/src/learn
Parameters: { "use_label_encoder" } are not used.

```

```

warnings.warn(msg, UserWarning)
/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [22:48:37] WARNING: /workspace/src/learn
Parameters: { "use_label_encoder" } are not used.

```

```
warnings.warn(smsg, UserWarning)
/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [22:50:15] WARNING: /workspace/src/learn
Parameters: { "use_label_encoder" } are not used.

warnings.warn(smsg, UserWarning)
/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [22:51:48] WARNING: /workspace/src/learn
Parameters: { "use_label_encoder" } are not used.

warnings.warn(smsg, UserWarning)
/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [22:53:20] WARNING: /workspace/src/learn
Parameters: { "use_label_encoder" } are not used.

warnings.warn(smsg, UserWarning)
/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [22:54:50] WARNING: /workspace/src/learn
Parameters: { "use_label_encoder" } are not used.

warnings.warn(smsg, UserWarning)
The most important 10 features ['recoveries', 'out_prncp_inv', 'out_prncp', 'funded_amnt', 'term', 'last_pymnt_d',
```

```
from sklearn.metrics import roc_auc_score
y_train_pred = model.predict_proba(x_train)[: , 1]
y_val_pred = model.predict_proba(x_val)[: , 1]

train_auc = roc_auc_score(y_train, y_train_pred)
val_auc = roc_auc_score(y_val, y_val_pred)

print(f"Train AUC: {train_auc:.4f}")
print(f"Validation AUC: {val_auc:.4f}")
```



```
-----
ValueError                                Traceback (most recent call last)
/tmp/ipython-input-21-3227849508.py in <cell line: 0>()
      3 y_val_pred = model.predict_proba(x_val)[: , 1]
      4
----> 5 train_auc = roc_auc_score(y_train, y_train_pred)
      6 val_auc = roc_auc_score(y_val, y_val_pred)
      7

----- 1 frames -----
/usr/local/lib/python3.11/dist-packages/sklearn/metrics/_ranking.py in roc_auc_score(y_true, y_score, average,
sample_weight, max_fpr, multi_class, labels)
    632 )
    633 if multi_class == "raise":
--> 634     raise ValueError("multi_class must be in ('ovo', 'ovr')")
    635 return _multiclass_roc_auc_score(
    636     y_true, y_score, labels, multi_class, average, sample_weight

ValueError: multi_class must be in ('ovo', 'ovr')
```

Next steps: [Explain error](#)

```
from sklearn.model_selection import RandomizedSearchCV
from xgboost import XGBClassifier

param_dist = {
    'max_depth': [3,5,7,9],
    'learning_rate': [0.01, 0.05, 0.1],
    'n_estimators': [100, 200, 300],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
}

xgb = XGBClassifier(use_label_encoder=False, eval_metric='logloss')

search = RandomizedSearchCV(xgb, param_dist, n_iter=20, scoring='roc_auc', cv=5, verbose=2, random_state=42)
search.fit(x_train, y_train)

print("Best params:", search.best_params_)
print("Best AUC:", search.best_score_)
```

```

from xgboost import XGBClassifier
best_params = search.best_params_
best_model = XGBClassifier(**best_params, use_label_encoder=False, eval_metric='logloss')
best_model.fit(x_train, y_train)

```

  -----

NameError Traceback (most recent call last)

[/tmp/ipython-input-22-3665359541.py](#) in <cell line: 0>()
 1 from xgboost import XGBClassifier
 ----> 2 best_params = search.best_params_
 3 best_model = XGBClassifier(**best_params, use_label_encoder=False, eval_metric='logloss')
 4 best_model.fit(x_train, y_train)

NameError: name 'search' is not defined

Next steps: [Explain error](#)

```

from sklearn.metrics import roc_auc_score, classification_report

```

```

y_val_pred_proba = best_model.predict_proba(x_val)[: , 1]
y_val_pred = best_model.predict(x_val)
print("Final AUC on validation set:", roc_auc_score(y_val, y_val_pred_proba))
print(classification_report(y_val, y_val_pred))

```

```

import pandas as pd
print(pd.Series(y_train).value_counts())

```

```

import numpy as np
unique_classes, counts = np.unique(y_train_encoded, return_counts=True)
print("Category and Sample size: ")
for cls, cnt in zip(unique_classes, counts):
    print(f"Category {cls}: {cnt}")

```

```

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

```

```

sns.set(style="whitegrid")
selected_features = [
    'recoveries', 'out_prncp_inv', 'out_prncp', 'funded_amnt', 'term',
    'last_pymnt_d', 'total_rec_prncp', 'last_pymnt_amnt',
    'debt_settlement_flag', 'hardship_flag'
]
df_selected = lending_club[selected_features].copy()
df_selected['term'] = df_selected['term'].str.extract('(\d+)').astype(float)

df_selected['last_pymnt_d'] = pd.to_datetime(df_selected['last_pymnt_d'], format='%b-%Y', errors='coerce')
df_selected['last_pymnt_d'] = df_selected['last_pymnt_d'].map(lambda x: x.toordinal() if pd.notnull(x) else np.nan)

df_selected['debt_settlement_flag'] = df_selected['debt_settlement_flag'].map({'Y': 1, 'N': 0})
df_selected['hardship_flag'] = df_selected['hardship_flag'].map({'Y': 1, 'N': 0})

```

```

corr_matrix = df_selected.corr()

```

```

sns.set()
plt.figure(figsize=(8, 6))

```

```

ax = sns.heatmap(
    corr_matrix,
    annot=True,
    fmt=".2f",
    vmin=-1, vmax=1, center=0,
    cmap="YlOrBr",
    square=True,
    linewidths=0.5,
    cbar_kws={"shrink": 0.75}
)

```

```
)

ax.set_xticklabels(
    ax.get_xticklabels(),
    rotation=45,
    horizontalalignment='right'
)

plt.title("Correlation Heatmap of Top 10 Important Features", fontsize=14)
plt.tight_layout()
plt.show()
```

```
corr_matrix = df_selected.corr()
print(corr_matrix)
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
loan_status_counts = pd.Series({
    'Fully Paid': 611631,
    'Current': 537982,
    'Charged Off': 152047,
    'Late (31-120 days)': 12788,
    'In Grace Period': 5354,
    'Late (16-30 days)': 2139,
    'Default': 19
})
```

```
df_plot = loan_status_counts.reset_index()
df_plot.columns = ['loan_status', 'count']
```

```
sns.set(style="whitegrid")
```

```
plt.figure(figsize=(8, 6))
ax = sns.barplot(
    data=df_plot,
    x='loan_status',
    y='count',
    color='#4c72b0',
    width=0.5
)
```

```
for index, row in df_plot.iterrows():
    ax.text(
        index, row['count'] + max(df_plot['count']) * 0.02,
        f"{row['count']:,}",
        color='black',
        ha='center',
        va='bottom',
        fontsize=10
    )
```

```
ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha='right')
```

```
plt.title("Loan Status Distribution", fontsize=14)
plt.xlabel("Loan Status")
plt.ylabel("Number of Loans")
```

```
plt.tight_layout()
plt.show()
```

```
#Cold Start
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
```

```
df_cold = lending_club[lending_club['loan_status'].isin(['Current', 'Issued'])].copy()
```

```
features = ['recoveries', 'out_prncp_inv', 'out_prncp', 'funded_amnt', 'term',
```

```

'total_rec_prncp', 'last_pymnt_amnt', 'debt_settlement_flag', 'hardship_flag']

valid_features = [col for col in features if (pd.to_numeric(df_cold[col], errors='coerce').isna().mean() <= 0.7)]
df_cold[valid_features] = df_cold[valid_features].apply(lambda col: pd.to_numeric(col, errors='coerce').fillna(0))

if 'term' in valid_features:
    df_cold['term'] = df_cold['term'].astype(str).str.extract('(\d+)').astype(int)
if 'debt_settlement_flag' in valid_features:
    df_cold['debt_settlement_flag'] = df_cold['debt_settlement_flag'].map({'Y':1, 'N':0})
if 'hardship_flag' in valid_features:
    df_cold['hardship_flag'] = df_cold['hardship_flag'].map({'Y':1, 'N':0})

x = df_cold[valid_features]

scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)

kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(x_scaled)
df_cold['cluster'] = clusters

print(df_cold.groupby('cluster')[valid_features].mean())
print(df_cold['cluster'].value_counts())

from sklearn.decomposition import PCA

pca = PCA(n_components=2, random_state=42)
x_pca = pca.fit_transform(x_scaled)

import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))
scatter = plt.scatter(x_pca[:,0], x_pca[:,1], c=clusters, cmap='viridis', alpha=0.6)
plt.xlabel('PCA 1')
plt.ylabel('PCA 2')
plt.title('KMeans Clustering Visualization (PCA 2D)')
plt.colorbar(scatter, label='Cluster')
plt.show()

#Cold Start
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

df_cold = lending_club[lending_club['loan_status'].isin(['Current', 'Issued'])].copy()

features = ['recoveries', 'out_prncp_inv', 'out_prncp', 'funded_amnt', 'term',
            'total_rec_prncp', 'last_pymnt_amnt', 'debt_settlement_flag', 'hardship_flag']

valid_features = [col for col in features if (pd.to_numeric(df_cold[col], errors='coerce').isna().mean() <= 0.7)]
df_cold[valid_features] = df_cold[valid_features].apply(lambda col: pd.to_numeric(col, errors='coerce').fillna(0))

if 'term' in valid_features:
    df_cold['term'] = df_cold['term'].astype(str).str.extract('(\d+)').astype(int)
if 'debt_settlement_flag' in valid_features:
    df_cold['debt_settlement_flag'] = df_cold['debt_settlement_flag'].map({'Y':1, 'N':0})
if 'hardship_flag' in valid_features:
    df_cold['hardship_flag'] = df_cold['hardship_flag'].map({'Y':1, 'N':0})

x = df_cold[valid_features]

scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)

kmeans = KMeans(n_clusters=6, random_state=42)
clusters = kmeans.fit_predict(x_scaled)
df_cold['cluster'] = clusters

print(df_cold.groupby('cluster')[valid_features].mean())
print(df_cold['cluster'].value_counts())

```



```
from sklearn.decomposition import PCA
pca = PCA(n_components=2, random_state=42)
x_pca = pca.fit_transform(x_scaled)

import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))
scatter = plt.scatter(x_pca[:,0], x_pca[:,1], c=clusters, cmap='viridis', alpha=0.6)
plt.xlabel('PCA 1')
plt.ylabel('PCA 2')
plt.title('KMeans Clustering Visualization (PCA 2D)')
plt.colorbar(scatter, label='Cluster')
plt.show()
```